

# Laboratorio di Calcolo

## A.A. 2026/2027, Canale A-Dec

**Giacomo Artoni**, Lorenzo Caprini

Primo programma in C

# Hello, World!

- Il programma più famoso della storia dell'informatica.
- Compare nel libro *The C Programming Language* di **Kernighan e Ritchie** (1978), il testo che ha diffuso il C.
- Da allora è il primo programma che si scrive in **ogni** nuovo linguaggio.
- Ci serve per studiare la **struttura** di un sorgente C, senza distrazioni.

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

```
$ gcc -Wall -Wextra -o hello hello.c
$ ./hello
Hello, World!
```

# Anatomia di un sorgente C

```c
/*
 * cerchio.c
 * Calcola la circonferenza di un cerchio
 */

#include <stdio.h>
#include <math.h>

int main() {
    double r = 1.5;
    double c;

    c = 2.0 * M_PI * r;
    printf("C = %f\n", c);

    return 0;
}
```

**1. Commento iniziale**

Nome del file e descrizione: lo leggono le persone, il compilatore lo ignora.

**2. Direttive al preprocessore**

Iniziano con `#`, includono le librerie (`stdio.h` per l'I/O, `math.h` per la matematica).

**3. La funzione `main`**

Contiene le istruzioni del programma: dichiarazioni, calcoli, output, valore restituito.

# La funzione main

- Ogni programma C contiene **una e una sola** funzione `main`.
- L'esecuzione **inizia** dalla prima istruzione di `main` e **termina** con `return`.
- `int`: tipo del valore che `main` restituisce al sistema operativo.
- `()`: elenco degli argomenti (qui nessuno; si può scrivere anche `int main(void)`).
- `{ ... }`: il **corpo** della funzione.
- `return 0;`: comunica che tutto è andato bene. Un valore diverso da zero segnala un errore.

```c
#include <stdio.h>

int main() {
    printf("Qualcosa e' andato storto\n");
    return 3;
}
```

```
$ ./errore
Qualcosa e' andato storto
$ echo $?
3
```

La variabile di shell `$?` contiene il valore restituito dall'ultimo programma eseguito.

# Istruzioni

Un'**istruzione** (*statement*) è il passo elementare di un programma.

- Ogni istruzione semplice termina con un **punto e virgola** `;`
- Le istruzioni vengono eseguite **in sequenza**, dall'alto verso il basso.
- Tipi di istruzioni che incontreremo:
  - **dichiarazioni:** `double t;`
  - **assegnamenti:** `t = 2.0;`
  - **chiamate di funzione:** `printf( ... );`
  - **controllo del flusso:** `if` , `for` , `while` (più avanti)
  - **ritorno:** `return 0;`

```c
int main() {
    double v0 = 10.0;       // dichiarazione + asse
    double t;               // dichiarazione
    t = 2.0;                // assegnamento
    printf("%f\n", v0 * t); // chiamata
    return 0;               // ritorno
}
```

Il `;` **termina** l'istruzione, non la separa: va messo anche dopo l'ultima.

# Blocchi

Le parentesi graffe `{ }` raggruppano più istruzioni in un **blocco** (o istruzione composta).

- Un blocco può stare ovunque sia ammessa un'istruzione.
- I blocchi si possono **annidare** uno dentro l'altro.
- Una variabile dichiarata in un blocco esiste **solo** all'interno di quel blocco (la sua *visibilità*).
- Ogni `{` deve avere la sua `}`: l'indentazione aiuta a verificarlo a colpo d'occhio.

```
1   #include <stdio.h>
2
3   int main() {
4       int a = 1;
5       {
6           int b = 2;
7           printf("a = %d, b = %d\n", a, b);
8       }
9       // qui b non esiste piu'
10      printf("a = %d\n", a);
11      return 0;
12  }
```

Usare `b` dopo la riga 8 produrrebbe `error: 'b' undeclared`. Rivedremo i blocchi con `if` e i cicli.

# Commenti

Testo ignorato dal compilatore, scritto per **chi legge** il codice (compresi voi tra un mese).

## Due sintassi

```c
/* Commento che puo' occupare
   piu' righe (stile C89) */

// Commento su una sola riga (C99)
double g = 9.81;  // anche a fine riga
```

I commenti `/* */` **non** si possono annidare: il primo `*/` chiude il commento.

## Commenti utili e inutili

```c
// INUTILE: ripete il codice
i = i + 1;   // incrementa i di 1

// UTILE: spiega significato e unita'
double dt = 0.01;  // passo temporale [s]

// UTILE: spiega il perche'
// Usiamo double: con float l'errore
// di arrotondamento cresce troppo
```

Un buon commento spiega il **perché** e il **significato**, non il **cosa**.

# Il C non guarda la formattazione

Spazi, tabulazioni e ritorni a capo sono (quasi) **irrilevanti** per il compilatore.

Questo programma compila e funziona esattamente come `hello.c`:

```c
#include <stdio.h>
int main(){printf("Hello, World!\n");return 0;}
```

E anche questo:

```c
#include <stdio.h>
int
main
(   )   {   printf  (
"Hello, World!\n"    )
;  return
0 ;   }
```

La formattazione serve alle **persone**: il codice si legge molte più volte di quante si scriva.

# Convenzioni di formattazione

## Difficile da leggere

```c
#include <stdio.h>
int main() {
double x=3.0,y;
  y=x*x+2*x-1;
      printf("%f\n",y);
return 0;}
```

## Leggibile

```c
#include <stdio.h>

int main() {
    double x = 3.0;
    double y;

    y = x * x + 2 * x - 1;
    printf("%f\n", y);

    return 0;
}
```

- **Indentazione coerente** (4 spazi) per ogni livello di blocco.
- **Un'istruzione per riga**, spazi attorno agli operatori, righe vuote tra le parti logiche.
- **Nomi significativi** e righe non troppo lunghe (≲ 80 caratteri).

# Parole chiave e identificatori

## Parole chiave (*keyword*)

Parole riservate dal linguaggio, che **non** si possono usare come nomi:

```
auto        break       case        char
const       continue    default     do
double      else        enum        extern
float       for         goto        if
int         long        register    return
short       signed      sizeof      static
struct      switch      typedef     union
unsigned    void        volatile    while
```

Le 32 parole chiave dello standard C89 (gli standard successivi ne aggiungono altre).

## Identificatori

Nomi scelti da noi per variabili e funzioni:

- lettere, cifre e `_`
- **non** possono iniziare con una cifra
- maiuscole ≠ minuscole: `massa` ≠ `Massa`

| Nome | |
|---|---|
| `velocita`, `v0`, `t_max` | **Sì** |
| `2pi` | **No!** Inizia con una cifra |
| `massa-totale` | **No!** `-` è un operatore |
| `int` | **No!** È una parola chiave |
| `velocità` | **No**: evitate gli accenti |

# Primo contatto con printf

`printf` (*print formatted*) scrive testo sul terminale. È dichiarata in `stdio.h`.

```c
#include <stdio.h>

int main() {
    int n = 42;
    printf("Prima riga\n");
    printf("Seconda ");
    printf("riga\n");
    printf("Il numero e' %d\n", n);
    return 0;
}
```

```
$ ./stampa
Prima riga
Seconda riga
Il numero e' 42
```

- Il testo va tra **virgolette doppie** `" … "`.
- `\n` (*newline*) va a capo: senza, si continua sulla stessa riga.
- `%d` è un **segnaposto**: viene sostituito dal valore di `n`.
- Vedremo i dettagli nel modulo sull'output formattato.

# Dal sorgente all'eseguibile: le quattro fasi

Il comando `gcc` esegue in sequenza quattro passaggi, che possiamo anche osservare uno alla volta.




- **Preprocessore:** esegue le direttive `#` (es. copia il contenuto di `stdio.h`) e rimuove i commenti.
- **Compilatore:** traduce il C in **Assembly** (vedi lezioni precedenti).
- **Assemblatore:** traduce l'Assembly in **codice macchina** (file oggetto `.o`).
- **Linker:** collega il file oggetto alle librerie (il codice di `printf`) e produce l'**eseguibile**.

# Errori tipici del primo programma

| Errore | Cosa dice `gcc` |
|---|---|
| `;` dimenticato | `error: expected ';' before 'return'` |
| `}` finale mancante | `error: expected declaration or statement at end of input` |
| Virgolette non chiuse | `warning: missing terminating " character` |
| `#include <stdio.h>` mancante | `warning: implicit declaration of function 'printf'`<br>`note: include '<stdio.h>' or provide a declaration of 'printf'` |
| `Main` invece di `main` | `undefined reference to `main'` (errore del **linker**) |

**Esercizio:** prendete `hello.c` funzionante e introducete **uno alla volta** questi errori. Leggete i messaggi del compilatore: imparerete a riconoscerli quando vi capiteranno davvero.